计算属性

在模板中可以直接通过插值语法显示一些data中的数据。但是在某些情况，我们可能需要对数据进行一些转化后再显示，或者需要将多个数据结合起来进行显示。

比如我们有firstName和lastName两个变量，我们需要显示完整的名称。

但是如果多个地方都需要显示完整的名称，我们就需要写多个{{firstName}} {{lastName}}

```
<h4>商品总价:{{total}}</h4>
var vm=  new Vue({
            el:'#app',
            data:{
                num:10,//数量
                price:12//单价
            },
            computed:{//计算属性
                total(){
                    console.log('计算属性触发了')
                    return this.num * this.price
                }
            }
        })
//使用方法计算data的值不变也会重新调用计算
//computed只会在data值发生变化时再次计算
```

getter和setter

每个计算属性都包含一个getter和setter,在上个例子中，我们只是用getter来读取。在某些情况下我们也可以提供一个setter方法（不常用）

计算属性实际上也是一个方法，它可以完成各种复杂的逻辑、包括运算，函数调用，最终返回一个值

计算属性是有缓存的，当值发生变化时才会再次计算

当你需要手动修改计算属性的值，你可以使用setter函数来修改

如果你只是想获取数据默认就是getter这函数来读取

```
 <div id="app">
        姓：<input type="text" v-model="firstName" /><br>
        名: <input type="text" v-model="lastName" />
        <h3>姓名：{{fullName}}</h3>
        <div style="border: 1px solid red;">
            <p>getter 与setter</p>
            姓名：<input type="text" v-model="fName" /><br>
            姓名(单项fullName)：<input type="text" v-model="fullName" /><br>
            <h3>{{fName}}</h3>
        </div>
```

```
    </div>
var vm=  new Vue({
        el:'#app',
        data:{
            firstName:"",
            lastName:""
        },
        computed:{//计算属性
            // 将姓名拼接
            fullName (){
                return this.firstName + '-' + this.lastName
            },
            fName:{
                get(){//默认是有get
            /*
              get 当需要读取当前属性的时候会根据相关的数据计算，并返回当前属性的值
             */
                  // 回调当前函数
                  return this.firstName + ' ' + this.lastName
                },
                set(v){
                    // set v:拿的是get返回的内容
                    // 监听当前属性值的变化，当属性值发生变化时执行，更新相关的属性数据
                    console.log(v)
                    var name =v.split(' ')
                    console.log(name)
                    this.firstName=name[0]
                    // 设置名
                    this.lastName=name[1]
                }
            }
        }
    })
```

侦听器

侦听器 watch 是 Vue 提供的一种用来观察和响应 Vue 实例上的数据变化的属性。当被侦听的数据发生变化时，会触发对应的侦听函数。

侦听器的使用

侦听器可以监听data对象属性或者计算属性的变化，watch是观察属性的变化，所以watch的属性名必须要与观察人的名字保持一致；这样观察的值发生了变化才会触发，

```
<div id="app">
    <h4>{{num}}</h4>
```

```
    <button type="button" @click="add">+</button>
    <button type="button" @click="reduce">-</button>
</div>

new Vue({
        el: '#app',
        data: {
            num: 0,
            isMax: false,//数量不超过10
        },
        methods: {
            add() {//将数量加+1
                console.log(this.isMax)
                if (this.isMax === true) {
                    return
                }
                this.num++;
            },
            reduce() {
                this.num--
            }
        },
        // watch 和methods，平级
        watch: {
            /*
                侦听num属性的数据变化
                作用：num的值一旦发生改变，这个方法就会被调用
                newVal, 新值
                oldVal：旧的值，之前的值
             */
            num(newVal, oldVal) {
                console.log(newVal, oldVal)
                // 当newVal<0时候
                if (newVal < 0) {
                    this.num = 0;
                } else if (newVal > 10) {
                    alert('库存不足')
                    this.num=10;
                    this.isMax=true;
                }
            }
        }
    })

    <div id="app">
        请输入你的姓名：<input type="text" v-model="users.name" />
```

```
        <h4>{{users}}</h4>
        <h4>{{names}}</h4>
    </div>
    new Vue({
        el:'#app',
        data:{
            users:{
                name:'12'
            },
            names:'aa'
        },
        watch:{
            //第一种方式：侦听整个对象，每个属性值变卦都会执行handler
            //handler 就是原来的侦听函数，当数据发生变化会执行该函数
            names:{
                handler(news,old){
                     console.log(news.name,old.name)
                    console.log(news)
                    immediate:true//监听一次
                    // immediate 代表watch里面声明了一次，理解为进入页面，就执行监听
函数handler
                    deep:true
                    // deep 属性代表是否深度监听，默认是false,当值为true的时候会对对象
每个属性进行侦听
                },
                // 第二种方式 ： 侦听对象的某一个属性,值发生变化就会执行
                'users.name' (news,old){
                  console.log(news,old)
            }
        }
    })
```

取消侦听

```
<div id="app">
        <h3>{{num}}</h3>
        <button type="button" @click="add">+</button>
        <button type="button" @click="reduce">-</button>
    </div>

    var vm= new Vue({
         el:'#app',
         data:{
             num:0
         },
         methods:{
            add(){
                this.num++
```

```
        },
         reduce(){
            this.num--
         }
     },
 console.log(vm)
 // 通过vue实例调用watch方法
var unwatch= vm.$watch("num",//侦听的数据(变量、属性、或者数组)
     function(newVal,oldVal){//当数据执行发生改变的回调函数
         console.log('new:',newVal)
         console.log('old:',oldVal)
         if(newVal<0){//当newval小于0的时候
               this.num=0;//num=0
            }
            // 回调unwatch
            //执行之后会取消侦听num
            unwatch()
     },
     {//设置的选项对象
          immediate:true,//取消侦听后使用侦听属性会报错
     }

 )
```